Tasmota 插座 MQTT 命令行脚本

cli\mqtt-6FCE2C-*.py 直跑脚本 + mqtt-cli.py 全参数脚本 · 使用说明

v3.2 · 2026-09-27 · 设备 tasmota_6FCE2C（固件 15.5.0 / ESP32-C2）
插座不在线时直接说明原因（broker 的遗嘱消息是 Offline），不再笼统报 timeout
新增 mqtt-cli.py：动作与全部设置都从命令行给，不读文件名

项目	内容
控制对象	Tasmota 插座：继电器开关 + 设备侧定时器
Broker	`s1.20202029.com` 1883 明文 / 8883 TLS 1.2（两端都需用户名+密码）
默认设备	`6FCE2C` → 主题 `tasmota_6FCE2C`
本册讲谁	`cli\mqtt-6FCE2C-*.py` 直跑脚本，外加不分动作的 `mqtt-cli.py`
工作目录	`D:\aiwork\mqtt-py\cli` —— 脚本和本册都在这里
Python	3.13（虚拟环境 `.venv`）
依赖	`paho-mqtt`；生成本 PDF 需要 `reportlab`；跑测试可选 `pytest`

目录

1 一分钟上手
2 环境与文件清单
3 继电器控制
4 定时任务（重点）
5 设备时钟与时区
6 换设备与快捷脚本
7 另一个脚本：mqtt-cli.py（全部参数都由命令行给）
8 参数速查
9 输出格式与退出码
10 常见问题排查
附录 A MQTT 主题映射
附录 B 本机实测注意事项

1 一分钟上手

这 11 个脚本就是**全部**：一个文件一个动作，双击或一行命令即可，文件名说明控制哪台设备、做什么。

命令都在 `D:\aiwork\mqtt-py\cli` 目录下执行，先准备解释器（激活虚拟环境，或用完整路径）：

```
cd D:\aiwork\mqtt-py\cli
..\.venv\Scripts\activate REM or call python by its full path:
..\.venv\Scripts\python.exe mqtt-6FCE2C-on.py
```

用法只有一条：运行一个直跑脚本 ——
文件名说明控制哪台设备、做什么。同一个文件改个名就能换动作或换设备（见 6.2）。

我想……	运行这个文件
开	`mqtt-6FCE2C-on.py`
关	`mqtt-6FCE2C-off.py`
切换（开□关）	`mqtt-6FCE2C-toggle.py`
查当前状态	`mqtt-6FCE2C-status.py`
开 5 秒后自动关	`mqtt-6FCE2C-pulse.py --seconds 5`
关 5 秒后自动开	`mqtt-6FCE2C-pulseoff.py`
关 120 秒后自动开	`mqtt-6FCE2C-pulseoff-120.py`
体检硬件/继电器配置	`mqtt-6FCE2C-diag.py`
看设备时钟与时区	`mqtt-6FCE2C-time.py`
列出全部定时器	`mqtt-6FCE2C-timers.py`
设一个定时器	`mqtt-6FCE2C-timer.py 3 --time 08:00 --days weekdays --action on --enable`
换一台设备	把文件名里的 `6FCE2C` 改成新设备 ID

三条最容易忽略的前提，先看这里能省掉大半排查时间：

> ① **定时器用「设备本地时间」**，而本机时区是固件默认的 `+01:00`，不是北京时间 —— 设 08:00 会在北京 15:00 触发。设定时器前先执行 `mqtt-6FCE2C-time.py --set-tz +8:00`。
> ② **定时器要过三道开关**：全局 `Timers` 开关 + 槽位 `Enable` + 时间到。本机全局开关初始是 `OFF`，只写槽位不会触发。
> ③ **星期掩码周日在最前面**：周一至周五是 `0111110`，写成 `1111100` 会变成周日到周四。

每个直跑脚本都是**自带全部代码的独立程序**（约 1200 行），单独拷走也能直接跑；换动作、换设备都只是改文件名（见 6.2）。

2 环境与文件清单

2.1 依赖

```
cd D:\aiwork\mqtt-py

# .venv already ships paho-mqtt + pytest; rebuild them with:
.venv\Scripts\python.exe -m pip install paho-mqtt

# or with uv:
uv venv && uv add paho-mqtt
```

2.2 文件清单

要跑的东西只有 `cli\` 一个目录：11 个直跑脚本 + 1 个不分动作的 `mqtt-cli.py` 躺在里面，本册也生成在同一个地方，整个目录拷到别的机器上照样能用。11 个直跑脚本正文逐字节相同、只靠文件名区分，所以别把它们和别的东西混在一起。

```
D:\aiwork\mqtt-py\ <- the project root
|-- cli\ <- the 11 command-line scripts (chapters 3-6)
| |-- mqtt-cli.py <- the all-arguments version (chapter 7)
| '-- the CLI manual is written here too
|-- build-cli\ <- the generator, only to rebuild
|-- .venv\ <- Python 3.13 environment
'-- .workbuddy\ <- project memory folder, keep it
```

要用的文件（都在 `cli\` 里）

文件	作用
`mqtt-6FCE2C-on.py`	开
`mqtt-6FCE2C-off.py`	关
`mqtt-6FCE2C-toggle.py`	切换
`mqtt-6FCE2C-status.py`	查状态
`mqtt-6FCE2C-pulse.py`	开 N 秒后自动关（默认 5 秒）
`mqtt-6FCE2C-pulseoff.py`	关 N 秒后自动开（默认 5 秒）
`mqtt-6FCE2C-pulseoff-120.py`	关 120 秒后自动开（秒数写在文件名里）
`mqtt-6FCE2C-diag.py`	硬件 / 继电器诊断
`mqtt-6FCE2C-timer.py`	读 / 写一个定时器槽位
`mqtt-6FCE2C-timers.py`	列出全部槽位 / 批量开关
`mqtt-6FCE2C-time.py`	看 / 改设备时钟与时区
`mqtt-cli.py`	**不靠文件名的版本**：动作与全部设置都从命令行给（见第 7 章）

上面 11 个直跑脚本**正文逐字节相同**，而且**各自独立**；文件名才是配置 —— 复制改名即可增减，单独拷一个走也能直接跑（见 6.2）。最后那个 `mqtt-cli.py` 与它们共用同一份引擎，但**从不读自己的文件名**，所以既可以单独用，也可以被改名或拷到别处（见第 7 章）。

工作目录里还有

文件	作用
`build-cli/`	**打造上面这些文件的工具，日常用不到**（见 2.3）
`.venv/`	虚拟环境（Python 3.13.12）
`.workbuddy/`	项目记忆目录，勿删

2.3 build-cli/ 里有什么

文件	作用
`build-cli/engine.py`	引擎源码 —— 每个直跑脚本的正文都是从它复制来的
`build-cli/make_shortcuts.py`	生成器：把引擎拼进每个脚本（`->cli\`）。改了引擎必须重跑它，否则改动到不了脚本里
`build-cli/selftest.py`	离线回归测试，不需要 broker（顺带校验脚本是否过期）
`build-cli/make_manual.py`	生成本 PDF 的脚本

> 跑自检：`python build-cli\selftest.py` 或 `pytest build-cli\selftest.py` —— 都不联网，秒级完成。

3 继电器控制

动作	说明
`on`	开。发 `cmnd/<topic>/POWER ON`，设备回 `stat/<topic>/POWER ON`
`off`	关，同上
`toggle`	取反 —— 不关心当前状态时用它
`status`	读状态（`Status 0` → `{"Status":{...,"Power":"1"}}`）
`pulse`	开 N 秒后自动关，N 由 `--seconds` 指定（默认 5）
`pulseoff`	`pulse` 的反面：关 N 秒后自动开，N 同样默认 5 秒
`diag`	诊断：固件版本、模板、GPIO 分配、继电器是否真的配好

```
mqtt-6FCE2C-on.py
mqtt-6FCE2C-off.py
mqtt-6FCE2C-toggle.py
mqtt-6FCE2C-status.py --json
mqtt-6FCE2C-pulse.py --seconds 5 on 5 s, then off
mqtt-6FCE2C-pulseoff.py off 5 s, then on
mqtt-6FCE2C-pulseoff-120.py off 120 s, then on
mqtt-6FCE2C-diag.py -v
```

两个脉冲动作的**秒数也能写进文件名**：文件名最后一段如果是数字，就当作 `--seconds`。所以 `mqtt-6FCE2C-pulseoff-120.py` 完全等于 `mqtt-6FCE2C-pulseoff.py --seconds 120`，双击就跑，不用带参数；命令行再给一个 `--seconds` 会覆盖文件名里的值。

只有 `pulse` 和 `pulseoff` 允许这么写，所以设备名自己带数字（例如 `mqtt-plug2-off.py`）不会被当成秒数。

`--relay` 用来选继电器：`0` = 全部继电器（发 `POWER0`），`1` = 第 1 路（发 `POWER`），`n` = 第 n 路。本机只有一路，一般不用管。

脚本会自动处理这台设备的一个历史怪癖：早期固件没注册「继电器 1」，`POWER` 会被直接拒绝，此时脚本自动改用 `POWER0` 重试，并在标准错误里打印一行 `[fallback] ...`。**看到这行不是错误**，命令已经成功。

4 定时任务（重点）

4.1 三道开关，缺一不可

层	是什么	怎么打开
全局	设备的 Timers 总开关	`mqtt-6FCE2C-timers.py --enable-all`
槽位	该槽位自己的 Enable 字段	设置时加 `--enable`
时间	设备本地时间到点	见第 5 章，先把时区改成 +8:00

> 本机全局开关出货状态是 OFF。只写槽位、不开全局开关，看起来「设置成功了」但永远不会触发 —— 这是最常见的坑。

4.2 一条命令写一个定时器

用直跑脚本 `mqtt-6FCE2C-timer.py`，骨架（槽位号 1~16 是**位置参数**，写在文件名后面）：

```
mqtt-6FCE2C-timer.py <slot> --time <time> --days <days> --action <action> --enable
```

例：3 号槽位，工作日 08:00 开，并启用该槽位：

```
mqtt-6FCE2C-timer.py 3 --time 08:00 --days weekdays --action on --enable
```

清单里没有这个文件时，随便复制一个脚本改名即可（正文无需改动）：

```
copy mqtt-6FCE2C-on.py mqtt-6FCE2C-timer.py
```

成功时的输出（设备会回显新值，脚本回读比对后才报 ok）：

```
action: timer slot 3 ok
timer3 : {"Action":1,"Days":"0111110","Enable":1,"Mode":0,
"Output":1,"Repeat":0,"Time":"08:00","Window":0}
sets : at 08:00, on Mon/Tue/Wed/Thu/Fri, -> output 1 ON, once, then disarms
```

写入策略是**先读该槽位现值、只合并你要改的字段、再回写完整 JSON**，所以不会碰你没提到的字段，对「要求字段齐全」的固件也安全。若设备没接受某个字段，会打印 `device stored different values: ...` 并返回退出码 1。

4.3 槽位字段

字段	命令行	含义与取值
Enable	`--enable` / `--disable`	槽位开关：1 启用 / 0 停用
Mode	`--mode`	时间基准：`clock` 固定时刻 / `sunrise` 日出 / `sunset` 日落
Time	`--time`	`clock` 模式写 `HH:MM`（00:00~23:59）；日出/日落模式写相对偏移 `+/-H:MM`，例 `-0:30`，小时最大 11
Window	`--window`	随机抖动分钟数 0~15，给触发时刻加随机偏移（避免所有设备同时动作）
Days	`--days`	7 位星期掩码，顺序 `SMTWTFS`，**周日在第 1 位**，详见 4.4
Repeat	`--repeat / --once`	`--repeat` 每天重复；`--once` 只触发一次并在触发后自动停用。两者都不给则该字段不变

字段	命令行	含义与取值
Output	`--output`	驱动哪一路输出/继电器，1~16，本机填 1
Action	`--action`	`off` 关 / `on` 开 / `toggle` 切换 / `rule` 触发规则 `Clock#Timer=<n>`

4.4 星期掩码 Days（最容易错的地方）

掩码共 7 位，**第 1 位是周日**，顺序固定 `S M T W T F S`；`0` 和 `-` 都表示「关」，其它字符表示「开」。

位序	1	2	3	4	5	6	7
代表	日	一	二	三	四	五	六
符号	S	M	T	W	T	F	S
周一~周五	0	1	1	1	1	1	0
周末	1	0	0	0	0	0	1
每天	1	1	1	1	1	1	1
错误写法	1	1	1	1	1	0	0

> 最后一行 `1111100` 是**周日~周四**，不是周一~周五。手写掩码时务必记住周日在最前；不确定就交给 `--days` 自己算。

`--days` 的易读写法（脚本替你换算成掩码）：

写法	结果掩码	说明
`weekdays`	`0111110`	周一至周五
`weekend`	`1000001`	周六与周日
`all`	`1111111`	每天（`everyday` / `daily` 同义）
`none`	`0000000`	不选任何一天
`Mon,Wed,Fri`	`0101010`	星期列表，逗号或空格分隔
`Mon-Fri`	`0111110`	区间，可跨周：`Fri-Mon` → `1100011`
`0111110`	`0111110`	直接给掩码，原样使用

大小写不敏感：`Mon,Wed,Fri` 与 `mon wed fri` 等价。

4.5 常用配方

场景	命令
工作日 08:00 开	`timer 3 --time 08:00 --days weekdays --action on --enable`
每天 23:00 关	`timer 4 --time 23:00 --days all --action off --enable`
一/三/五 18:30 开	`timer 5 --time 18:30 --days Mon,Wed,Fri --action on --enable`
日落前 30 分钟开	`timer 6 --mode sunset --time -0:30 --action on --enable`
日出后 15 分钟关	`timer 7 --mode sunrise --time +0:15 --action off --enable`
每天 07:00 开，随机 ±10 分钟	`timer 8 --time 07:00 --days all --action on --window 10 --enable`
只触发一次，触发后自动停用	`timer 9 --time 06:00 --days all --action on --once --enable`
只改动作，其它字段不动	`timer 3 --action toggle`
停用槽位	`timer 3 --disable`
清空槽位	`timer 3 --clear`

> 共 16 个槽位。建议先排好「哪个槽位干什么」再逐条写，比事后从 `timers` 的输出里反推要清楚得多。

4.6 查看、批量开关与清空

命令	作用
`timers`	列出全部槽位 + 全局开关状态
`timers --enable-all`	全局 `Timers` 置 ON（定时器总开关）
`timers --disable-all`	全局 `Timers` 置 OFF
`timers --toggle-all`	翻转全局开关
`timer 3 --clear`	清空 3 号槽位（Enable=0、Time=00:00、Days=0000000）

`timers` 的真实输出（本机）：

```
global : OFF <- no timer fires until you run: timers --enable-all
slots : 1 of 16 configured
idx arm mode time win days(SMTWTFS) rep out action (days)
3 1 clock 08:00 0 0111110 0 1 on (Mon/Tue/Wed/Thu/Fri)
```

列含义依次为 `idx` 槽位号 · `arm` 槽位开关 · `mode` 时间基准 · `time` 触发时间 · `win` 随机抖动 · `days` 星期掩码 · `rep` 是否重复 · `out` 输出号 · `action` 触发动作。第一行 `global` 是全局开关，看到 `OFF` 就去跑 `timers --enable-all`。

5 设备时钟与时区

定时器按设备本地时间触发，所以设定时器前必须先看时钟。`mqtt-6FCE2C-time.py` 读取 `Status 7`，把设备 UTC、本地时间、实际生效偏移量和对应北京时间一起算给你看：

```
mqtt-6FCE2C-time.py

action: time device clock (from `Status 7`)
utc : 2026-09-25T09:44:29Z
device loc : 2026-09-25T10:44:29
timezone : +01:00 (applied offset UTC+1)
beijing : 2026-09-25T17:44:29
sunrise : 06:41 sunset: 18:42
note: device local time is UTC+1, i.e. 7.0 h behind Beijing. `Timer` times are DEVICE local
time,
so a 08:00 timer would fire at 15:00 Beijing.
note: fix it (China has no DST, so a fixed offset is fine): mqtt-6FCE2C-time.py --set-tz +8:00
```

修正时区（中国没有夏令时，写固定偏移即可）：

```
mqtt-6FCE2C-time.py --set-tz +8:00
```

> 注意：**本机固件只认「+8:00」这种写法**：写 `+08:00`（小时带前导 0）会被**静默拒绝** —— 设备回一个格式完全正常的 `{"Timezone":"+00:00"}`，看不出报错，时区却回落成 UTC。实测对照：`+8` / `+8:00` / `8` 通过，`+08:00` 与 `+08` 失败。`--set-tz` 现在会自动把 `+08:00` 换算成 `+8:00` 并**回读校验**，失败会明确报错而不是假装成功。

> `+8:00` 表示「固定 UTC+8」。若写成 `99`，设备会改用 `TimeSTD/TimeDST` 的夏令时规则，中国不需要，别用。改完再跑一次 `mqtt-6FCE2C-time.py`，确认 `device loc` 已变成 UTC+8。

时钟由 NTP 同步，实测设备 UTC 与真实 UTC 秒级一致，所以只要时区对了，定时器就是准的。

6 换设备与快捷脚本

6.1 改设备名（三种方式，任选）

脚本正文里**没有**「设备名」这个常量 —— 设备完全由文件名决定，所以下面三种方式互不干扰。

方式	怎么做	优先级
改文件名	把直跑脚本名里的设备段换掉（见 6.2）—— 最省事，正文一个字都不用动	**推荐**
环境变量	`set MQTT_DEVICE=A1B2C3`（cmd）；`setx` 可永久保存	中
命令行	`--device A1B2C3` 或 `-d A1B2C3` 或 `--topic tasmota_A1B2C3`：只影响这一次运行，并**覆盖文件名**	最高

这些写法都指向同一台设备：`6FCE2C`、`DVES_6FCE2C`、`0x6FCE2C`、`tasmota_6FCE2C`；自定义主题名（如 `kitchen_plug`）原样使用。短 chip id 自动补零到 6 位：`1A` → `tasmota_00001A`。

6.2 直跑脚本：文件名就是配置

这些脚本都在 `cli\` 目录里。全部直跑脚本的正文**逐字节完全相同**：引擎代码已经内联在每个文件里，设备和动作则在运行时从自己的文件名读出 —— **改名即换设备、改名即换动作**，而且每个文件都是独立完整的程序：

文件名	控制谁	做什么
`mqtt-6FCE2C-on.py`	`tasmota_6FCE2C`	开
`mqtt-A1B2C3-off.py`	`tasmota_A1B2C3`	关
`mqtt-kitchen_plug-on.py`	`kitchen_plug`	开
`mqtt-6FCE2C-status.py`	`tasmota_6FCE2C`	查状态
`mqtt-6FCE2C-timer.py 3 ...`	`tasmota_6FCE2C`	写 3 号定时槽

每一个动作都能做成文件（这就是全部十个动作）：`on off toggle status pulse pulseoff diag timers timer time`。需要参数的动作把参数写在文件名后面，例如 `mqtt-6FCE2C-timer.py 3 --time 08:00 --days weekdays --action on --enable`。

命名规则 `mqtt-<device>-<action>.py`（`<device>` 处填设备、`<action>` 处填动作）：去掉扩展名后，按**最后一个** `-` 或 `_` 切开 → （设备, 动作），再剥掉开头装饰性的 `mqtt-`。动作大小写不敏感，必须是上面十个之一，否则脚本会退出码 1 并提示正确格式。

- 设备名自身可含分隔符：`mqtt-kitchen-plug-off.py` → 设备 `kitchen-plug`、动作 `off`
- 旧写法仍兼容：`6FCE2C_on.py` 等同 `mqtt-6FCE2C-on.py`
- 文件名前缀 `mqtt-` 只是装饰，可省：`6FCE2C-time.py` 同样指向 `tasmota_6FCE2C`
- 参数照旧透传：`--device OTHER` 临时改指别的设备，`--json` 输出机器可读结果，`-v` 打印全部报文，`--help` 看完整参数表
- **秒数可以写进文件名**：`pulse` / `pulseoff` 的名字最后一段若是数字就当作 `--seconds`，例如 `mqtt-6FCE2C-pulseoff-120.py` 等于 `mqtt-6FCE2C-pulseoff.py --seconds 120` —— 双击即跑，命令行再给 `--seconds` 则以命令行为准
- **每个脚本都能单独拷走**：引擎已内联在文件里，运行时不需要同目录的任何文件，拷到 U 盘或别的电脑照样跑

从零生成一整套（十个动作 + 文件名带秒数的那几个，共 11 个文件），并且**每次都顺手写出** `mqtt-cli.py`（第 7 章那个全参数版本）；也可以只生成你要的几个。**脚本一律写进** `cli\`：

```
python build-cli\make_shortcuts.py # -> cli\mqtt-6FCE2C-*.py (11 files)
python build-cli\make_shortcuts.py A1B2C3 # -> cli\mqtt-A1B2C3-*.py (11 files)
python build-cli\make_shortcuts.py kitchen_plug # a custom topic name works too
python build-cli\make_shortcuts.py A1B2C3 --only on,off,timer
```

日常更省事的做法是**留一个、复制改名**：

```
copy mqtt-6FCE2C-on.py mqtt-6FCE2C-timer.py
copy mqtt-6FCE2C-on.py mqtt-A1B2C3-on.py
```

7 另一个脚本：mqtt-cli.py（全部参数都由命令行给）

前面那些直跑脚本靠**文件名**表达「控制谁、做什么」，所以最省事，但也有它写不下东西：文件名塞不进一个主题名、一个继电器号、一个定时槽的星期掩码或一个时区。`mqtt-cli.py` **就是为这些情况准备的**：它和直跑脚本是同一个引擎、同一天生成，唯一区别是**它从不看自己的文件名** —— 动作是第一个参数，其它全部是开关，所以同一个文件可以驱动任意设备、做任意动作。

7.1 什么时候用它

情况	用哪个
开、关、查状态、切换，就是固定那台插座	`mqtt-6FCE2C-on.py` 等直跑脚本（双击就行）
临时换台设备（不想改名、也不想留痕迹）	`mqtt-cli.py on --device A1B2C3`
写脚本 / 批处理里调用	`mqtt-cli.py off --json`，一个文件走天下
要设主题、继电器号、时区、定时槽的星期	`mqtt-cli.py timer 3 --days weekdays ...`
把工具拷到别的电脑、名字被改过了	`mqtt-cli.py`：名字无关紧要

两者的关系：同一天由同一个引擎生成，把中间那段引擎代码拿出来的话**逐字节相同**，所以引擎的修正会同时到两者身上。直跑脚本 = 名字即配置；`mqtt-cli.py` = 参数即配置。

7.2 动作必填，写法与直跑脚本一致

动作是**第一个位置参数**，十个动作一个不少（`on off toggle status pulse pulseoff diag timers timer time`）。不写动作会直接报错退出（退出码 2），**不会**替你猜一个。

```
python mqtt-cli.py on
python mqtt-cli.py off --device A1B2C3
python mqtt-cli.py toggle -d kitchen_plug
python mqtt-cli.py status --json
python mqtt-cli.py pulse --seconds 30
python mqtt-cli.py pulseoff --seconds 120
```

`pulse` 与 `pulseoff` 的秒数只能写在命令行上（文件名不再参与），默认仍是 5 秒。

7.3 定时器与时钟

槽位号跟在动作后面，其余开关与第 4、5 章完全一样：

```
python mqtt-cli.py timer 3 --time 08:00 --days weekdays --action on --enable
python mqtt-cli.py timer 3 --time 18:30 --days Mon,Wed,Fri --action off --enable
python mqtt-cli.py timer 3 --mode sunset --time -0:30 --action on --enable
python mqtt-cli.py timer 3 --clear
python mqtt-cli.py timers
python mqtt-cli.py timers --enable-all
python mqtt-cli.py time --set-tz +8:00
```

> 定时器要靠「设备本地时间」，而设备的时区是固件默认的 `+01:00`。在设任何定时之前先跑一次 `python mqtt-cli.py time --set-tz +8:00`，否则 08:00 会在北京的 15:00 触发。

7.4 连接参数一样可以用命令行给

第 8.2 节列出的全部连接开关对 `mqtt-cli.py` 同样有效：

```
python mqtt-cli.py status --host 192.168.1.10 --port 1883
python mqtt-cli.py status --user mqtt --password SECRET
python mqtt-cli.py status --tls --port 8883 --ca server-47.112.1.248\certs\ca.crt
python mqtt-cli.py on -d 6FCE2C --timeout 10 -v
```

完整开关列表用 `python mqtt-cli.py --help` 查看（与直跑脚本共用同一张参数表，共 36 个开关）。

7.5 换个名字照样跑

因为这个文件不读自己的名字，把它改名或拷走都不影响行为：

```
copy mqtt-cli.py D:\tools\plug.py
python D:\tools\plug.py off --device A1B2C3
```

唯一的差别在**提示文案**：直跑脚本会提示「运行 `mqtt-6FCE2C-timers.py --enable-all`」，提示语里那个文件确实存在；`mqtt-cli.py` 则提示「`python mqtt-cli.py timers --enable-all`」，因为命令行版本旁边并没有一个叫 `mqtt-6FCE2C-timers.py` 的文件。

8 参数速查

8.1 动作与通用参数

下表的 <action> 就是**文件名里的动作段**：写成 mqtt-6FCE2C-<action>.py，参数直接跟在脚本名后面。例如 mqtt-6FCE2C-timer.py 3 --time 08:00 --days weekdays --action on。mqtt-cli.py 里 action 则是第一个位置参数，其余用法相同（见第 7 章）。

参数	取值 / 默认	说明
action	on off toggle status pulse pulseoff diag timers timer time	文件名里的动作段，十个之一（不是命令行参数）
index	1~16	位置参数，timer / timers 的槽位号，等同 --timer-index
--relay	默认 1	0=全部（POWER0），1=POWER，n=POWERn
--seconds	默认 5.0	pulse 保持 ON 的秒数；pulseoff 是保持 OFF 的秒数 —— 两者都能写进文件名
--timeout	默认 6.0	等待设备回复的秒数
--grace	默认 0.6	订阅后收集保留消息的等待时间
--retain	关闭	给命令加 retain 标记（一般别开，见附录 B）
--json	关闭	输出机器可读的 JSON
-v / --verbose	关闭	把每条 MQTT 收发报文打到标准错误

8.2 连接与设备参数

参数	对应环境变量	默认值
--host	MQTT_HOST	s1.20202029.com
--port	MQTT_PORT	1883
--user	MQTT_USER	mqtt
--password	MQTT_PASS	脚本内置
--tls	MQTT_TLS	关闭；开启后端口默认改 8883
--ca	MQTT_CA	server-47.112.1.248\certs\ca.crt
--device / -d	MQTT_DEVICE	6FCE2C
--chip-id	MQTT_CHIP_ID	旧别名，兼容用
--topic	—	整条主题，原样使用，优先级最高
--client-id	—	默认随机；**切勿填设备 ID**（见附录 B）

走 TLS 的例子：

```
mqtt-6FCE2C-status.py --tls --port 8883 --ca server-47.112.1.248\certs\ca.crt
```

9 输出格式与退出码

人读输出是两行；加 `--json` 变成一行 JSON，便于脚本解析：

```
> mqtt-6FCE2C-off.py
action: off state: OFF online: yes

> mqtt-6FCE2C-off.py --json
{"action": "off", "device": "6FCE2C", "note": null, "ok": true, "online": true,
"source": "stat/tasmota_6FCE2C/STATUS", "state": "OFF", "topic": "tasmota_6FCE2C"}
```

码	含义	典型原因
0	成功	命令执行且回读一致
1	错误	设备拒绝命令、参数非法、连接被拒、状态与期望不符
2	超时	设备没回复：离线、主题写错、网络不通
3	缺少依赖	`paho-mqtt` 没装

批处理里判断 `%ERRORLEVEL%` 即可。注意「设备拒绝命令」返回 1 而不是 2 ——
超时与被拒分得清楚，便于区分「设备不在线」和「命令不被接受」。

> 报「状态不符」之前，脚本会自动补一次 `Status 0` 权威回读。因为设备每 300
> 秒会主动推一条状态消息，它可能夹在命令与回显之间带着旧值，直接比较会造成假的「命令没生效」。

10 常见问题排查

现象	原因	处理
打印 `[fallback] POWER rejected ...` 后正常完成	固件没注册「继电器 1」，`POWER` 被拒	正常现象，脚本已改用 `POWER0` 成功执行，无需处理
打印 `the plug is OFFLINE ...`（退出码 2）	`tele/<topic>/LWT` 说 Offline：插座自己断电、掉 Wi-Fi 或死机了，不是本机网络或密码的问题	给插座断电 10 秒再插上；它自己那个网页（`Status 5` 查到的 IP）能打开，就说明已经回到网里了
`timeout: no reply to Status 0`，退出码 2	设备离线、主题不对或网络不通	先 `status -v` 看 `LWT`；再核对 `--device`
`rejected by the device: {"Command":"Error" ...}`	该固件不接受这条命令	加 `--relay 0` 强制走全部继电器；或跑 `diag` 看硬件配置
`device reports ON after asking for OFF`	继电器没真正切换；也可能插座里开着的规则（Rule）或定时器又把继电器翻回去了	脚本已自动补读；跑 `diag` 查模板/GPIO；再到插座自己的网页里把还开着的规则 / 定时器关掉，然后重试
定时器设好却不触发	三道开关没全开，或时区不对	`timers` 看全局是否 ON、槽位是否 `--enable`；`time` 确认本地时间
设 08:00 却在 15:00 触发	设备时区是 UTC+01:00	先跑 `time --set-tz +08:00`
`cannot tell the device and the action from ...`	文件名不符合 `mqtt-<device>-<action>.py`	改名为 `mqtt-6FCE2C-off.py` 这种形式；或者干脆用 `mqtt-cli.py off` 把动作写在参数里
`mqtt-cli.py` 报缺少 action	这个版本的动作是**必需的位置参数**，不从文件名猜	写成 `mqtt-cli.py on` —— 动作放最前面
`broker refused the connection (rc=5)`	用户名或密码不对	检查 `--user` / `--password`
paho-mqtt is not installed，退出码 3	依赖缺失	`pip install paho-mqtt`

附录 A MQTT 主题映射

Tasmota 的 `FullTopic` 是 `%prefix%/%topic%/`，默认主题 `tasmota_%06X`，其中 `%06X` 是设备开机打印的 chip id。本机 `DVES_6FCE2C` → `tasmota_6FCE2C`。

方向	主题	载荷 / 说明
发	`cmnd/<topic>/POWER`	`ON / OFF / TOGGLE`
发	`cmnd/<topic>/POWER0`	全部继电器；早期固件的回退路径
发	`cmnd/<topic>/Status`	`0` 查继电器位掩码
发	`cmnd/<topic>/Status`	`7` 查时钟与时区
发	`cmnd/<topic>/Timer<n>`	定时器 JSON，n = 1~16
发	`cmnd/<topic>/Timers`	`0/1/2` 全局关/开/翻转；空载荷 = 列出
发	`cmnd/<topic>/Timezone`	`+8:00`（本机固件拒绝 `+08:00`）
收	`stat/<topic>/RESULT`	命令回执，错误也走这里
收	`stat/<topic>/POWER`	`ON / OFF`
收	`stat/<topic>/STATUS`	`{"Status":{...,"Power":"1"}}`
收	`stat/<topic>/STATUS11`	`{"StatusSTS":{...,"POWER":"ON"}}`
收	`tele/<topic>/LWT`	`Online / Offline`（保留消息）

附录 B 本机实测注意事项

以下都是 2026-09-25
在真机上验证到的、与通用文档不同的地方。脚本已全部处理，记在这里是为了排查时不被误导。

- **状态怎么读**：用 `Status 0`，取 `{"Status":{...}}` 里的 `Power`。**不要**依赖保留消息 —— 设备的 `PowerRetain/StateRetain/StatusRetain` 全为 0，状态类主题不保留，必须主动查询；只有 `tele/.../LWT` 是保留的。
- **别把能耗当状态**：`StatusSNS.ENERGY.Power` 是「瓦特」，和继电器位掩码 `Power` 同名。脚本优先找显式的 `POWER / POWER<n>` 键，且不进入 `ENERGY` 子树，否则 0 瓦会被误读成「继电器 OFF」。
- **千万别用设备的 Client ID**：必须用随机 ID（脚本生成 `wb-mqtt-py-xxxxxx`），填 `DVES_6FCE2C` 会把插座自己顶下线。
- **命令默认不带 retain**：带上后插座每次重连 broker 都会重放最后一条命令，可能莫名其妙自己开关。只有要复刻设备侧设置时才加 `--retain`。
- **设备会主动推状态**：`TelePeriod` 为 300 秒，期间可能来一条 `STATUS11` 带着命令前的旧值；脚本判定失败前会补读一次，避免误报。
- **设备配置当天变过**：早期继电器完全没配（所有 GPIO 未分配、模板无 Relay），只有 `POWER0` 被接受且无回显；重启后继电器配好，`POWER` 正常工作并回显。脚本两条路径都兼容：先发 `POWER`，被拒才回退 `POWER0`。
- **时钟**：NTP 已同步（设备 UTC 与真实 UTC 秒级一致），但时区是固件默认 `+01:00`，需手动改成 `+8:00`。
- **时区写法的坑**：`Timezone +08:00` 会被**静默拒绝**（回 `+00:00`，时区却回落 UTC），必须写 `+8:00`。脚本现在两种写法都收，自动换算并回读 `Status 7` 校验。

— 文档结束 —